



Universidad Autónoma de Querétaro
Facultad De Ingeniería
Maestría En Ciencias en Inteligencia
Artificial



**FACULTAD DE
INGENIERÍA**

Problema de Empaquetado en Contenedores con Colonia de Hormigas

Algoritmos Metaheurísticos Tarea 6

P R E S E N T A

Ing. Daniel Eduardo González Alvarado

Profesor

Dr. Marco Antonio Aceves Fernández

01 de Diciembre de 2025

Índice

1. Introducción	1
1.1. Bin Packing Problem (BPP)	1
1.2. Técnicas de optimización inspiradas en procesos naturales	1
1.3. Ant Colony Optimization aplicada al Bin Packing	1
1.4. Problema	3
2. Desarrollo	4
2.1. Consideraciones iniciales	4
2.2. Codificación	4
2.3. Inicialización de feromonas y heurística	4
2.4. Construcción de soluciones por las hormigas	5
2.5. Evaluación de la función de aptitud	6
2.6. Actualización de feromonas	7
2.7. Ejecución del algoritmo y recolección de métricas	8
3. Resultados	9
4. Discusión	9
5. Conclusión	9
6. Bibliografía	10

1. Introducción

1.1. Bin Packing Problem (BPP)

El Bin Packing Problem (BPP), o problema de empaclado en contenedores, consiste en asignar un conjunto de objetos (paquetes) de diferentes tamaños buscando un número mínimo de contenedores con capacidad limitada. Tanto los objetos como los contenedores se consideran homogéneos: cada objeto posee un tamaño $p_i > 0$ y cada contenedor tiene una capacidad fija C . Debido a la naturaleza combinatoria del problema, el BPP es considerado NP-hard, por lo que resolverlo de manera exacta se vuelve inviable a medida que el número de objetos crece.

1.2. Técnicas de optimización inspiradas en procesos naturales

En las últimas décadas, muchos problemas complejos de optimización han sido abordados mediante algoritmos inspirados en procesos naturales. Estos métodos imitan comportamientos observados en la biología, la física y la ecología, permitiendo explorar grandes espacios de búsqueda de forma eficiente sin requerir una solución exacta.

Entre las técnicas más comunes se encuentran:

- **Optimización por Enjambre de Partículas (PSO):** Modela el movimiento colectivo de bandadas de aves o bancos de peces, actualizando las soluciones mediante la experiencia individual y colectiva.
- **Recocido Simulado (SA):** Inspirado en el proceso físico de enfriamiento del metal, explora soluciones aleatorias permitiendo aceptar temporalmente soluciones peores para evitar óptimos locales.
- **Sistemas Inmunes Artificiales (AIS):** Simulan la adaptación del sistema inmunológico, identificando patrones, clonando buenas soluciones y descartando respuestas ineficientes.

Estas técnicas, conocidas como metaheurísticas, son especialmente útiles para problemas NP-hard, donde la búsqueda exhaustiva es impráctica. Entre ellas, uno de los métodos más efectivos para problemas combinatorios es la Ant Colony Optimization (ACO).

1.3. Ant Colony Optimization aplicada al Bin Packing

El método de Ant Colony Optimization (ACO), propuesta originalmente por Dorigo y colaboradores [1, 2], es una metaheurística inspirada en la forma en que las hormigas reales encuentran rutas óptimas entre su colonia y una fuente de alimento. Su mecanismo central se basa en el uso de feromonas, una señal química que marca rutas prometedoras; conforme más hormigas utilizan un camino, mayor es la cantidad de feromona acumulada, reforzando su probabilidad de ser elegido nuevamente.

En el contexto del Bin Packing Problem (BPP), cada hormiga construye una solución asignando objetos a contenedores siguiendo un proceso probabilístico que combina dos fuentes de información:

1. **La feromona** almacenada para cada combinación objeto–contenedor, que refleja qué tan útil ha sido esta asignación en soluciones previas.

2. **La heurística**, que evalúa localmente la conveniencia de colocar un objeto en un contenedor, típicamente basada en el espacio restante.

La probabilidad de que una hormiga k asigne el objeto i al contenedor j viene dada por:

$$P_{ij}^{(k)} = \frac{\tau_{ij}^{\alpha} \eta_{ij}^{\beta}}{\sum_{l \in \text{Contenedores válidos}} \tau_{il}^{\alpha} \eta_{il}^{\beta}}, \quad (1)$$

donde:

- τ_{ij} es la cantidad de feromona asociada a asignar el objeto i en el contenedor j .
- η_{ij} es la heurística, por ejemplo:

$$\eta_{ij} = \frac{1}{1 + \text{espacio restante en el contenedor } j} \quad (2)$$

favoreciendo contenedores cuyo espacio sobrante sea pequeño (mejor llenado).

- α controla la **importancia de la feromona**.
- β controla la **importancia de la heurística**.

Evaporación y depósito de feromonas

Las feromonas evolucionan dinámicamente en cada iteración. Primero ocurre la **evaporación**, que evita que soluciones viejas dominen indefinidamente:

$$\tau_{ij} \leftarrow (1 - \rho) \tau_{ij}, \quad (3)$$

donde ρ es la *tasa de evaporación*. En nuestro caso, $\rho = 0,5$ implica que la mitad de la feromona se pierde en cada iteración, lo que mantiene la exploración activa.

Después de que todas las hormigas han construido sus soluciones, se deposita feromona adicional en las asignaciones que pertenecen a la mejor solución de la iteración o a la mejor global:

$$\tau_{ij} \leftarrow \tau_{ij} + \Delta\tau_{ij}, \quad (4)$$

donde la cantidad añadida sigue el esquema clásico:

$$\Delta\tau_{ij} = \begin{cases} \frac{Q}{f_{\text{best}}}, & \text{si el objeto } i \text{ fue asignado al contenedor } j \text{ en la mejor solución,} \\ 0, & \text{en otro caso,} \end{cases} \quad (5)$$

y f_{best} es el valor de la función de aptitud de la mejor hormiga. El parámetro Q determina cuánta feromona se deposita; al usar $Q = 1,0$, el refuerzo es proporcionalmente moderado y estable.

Interpretación en el Bin Packing

En este modelo:

- Una asignación que contribuye a llenar mejor los contenedores recibe más feromona.
- La heurística ($\beta = 2,0$) tiene mayor peso que la feromona, favoreciendo decisiones inmediatas basadas en el espacio restante.
- La evaporación ($\rho = 0,5$) mantiene un equilibrio entre exploración y explotación.

Así, las hormigas no solo siguen caminos exitosos, sino que también exploran nuevas combinaciones de asignación, permitiendo gradualmente encontrar configuraciones más eficientes de empaclado.

1.4. Problema

Para este trabajo considerademos 5 contenedores, de la A a la E, cada uno con capacidad 6 y 7 paquetes con tamaños variados que deben ser asignados a estos contenedores.

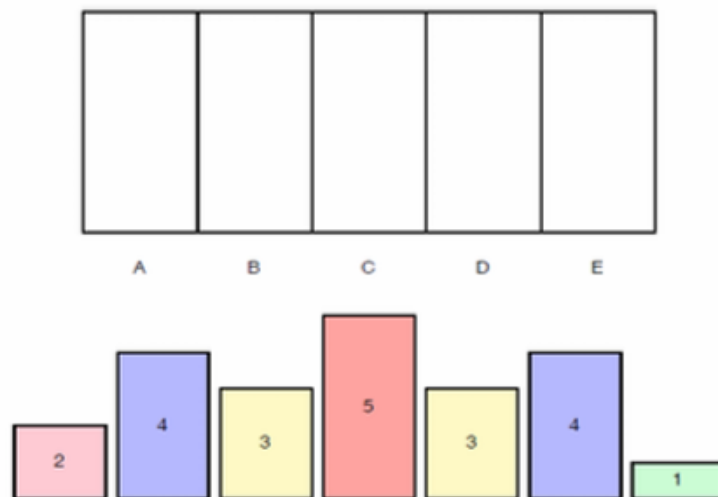


Figura 1: Representación del problema de empaclado en contenedores.

Ejemplos de forma de acomodar los paquetes en los contenedores.

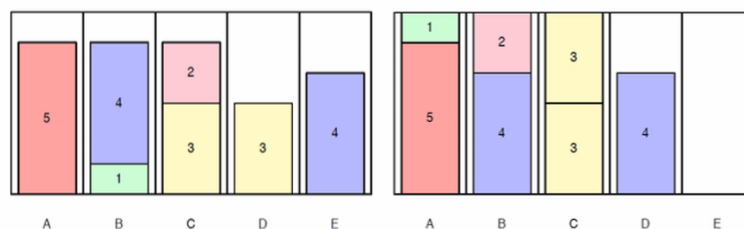


Figura 2: Ejemplos de formas de acomodar los paquetes en los contenedores.

2. Desarrollo

Para nuestro desarrollo partimos de un punto importante, como podemos ver en la Figura 2 la capacidad maxima de nuestros contenedores es de 6 unidades, lo que debemos tomar en cuenta al momento de asignar los paquetes, ya que no podemos sobrepasar esta capacidad.

2.1. Consideraciones iniciales

Primeramente se define la instancia del problema y los parámetros generales del algoritmo. Se especifican:

- El conjunto de objetos a empacar:

$$\mathbf{p} = [2, 4, 3, 5, 3, 4, 1],$$

donde cada dato corresponde al tamaño de un paquete como se ve en la Figura 1.

- La capacidad de los contenedores, $C = 6$, así como el número de contenedores disponibles.
- Los parámetros del algoritmo de colonia de hormigas:

$$\alpha = 1,0, \quad \beta = 2,0, \quad \rho = 0,5, \quad Q = 1,0,$$

2.2. Codificación

Cada solución se representa como un vector donde la posición indica el objeto y el valor indica el contenedor asignado. Por ejemplo:

$$\mathbf{s} = [A, A, B, C, B, D, C]$$

indica que el primer y segundo objeto van al contenedor A, el tercero y quinto al B, el cuarto y séptimo al C, etc. Es decir esta representación se maneja como una lista o arreglo de enteros (índices de contenedor).

Con lo cual debemos calcular para cada contenedor:

- La suma de tamaños de los objetos asignados (carga).
- Si se viola o no la capacidad máxima C .

2.3. Inicialización de feromonas y heurística

Se inicializa una matriz de feromonas τ_{ij} de tamaño

$$(\text{número de objetos}) \times (\text{número de contenedores}),$$

es decir,

$$(7) \times (5),$$

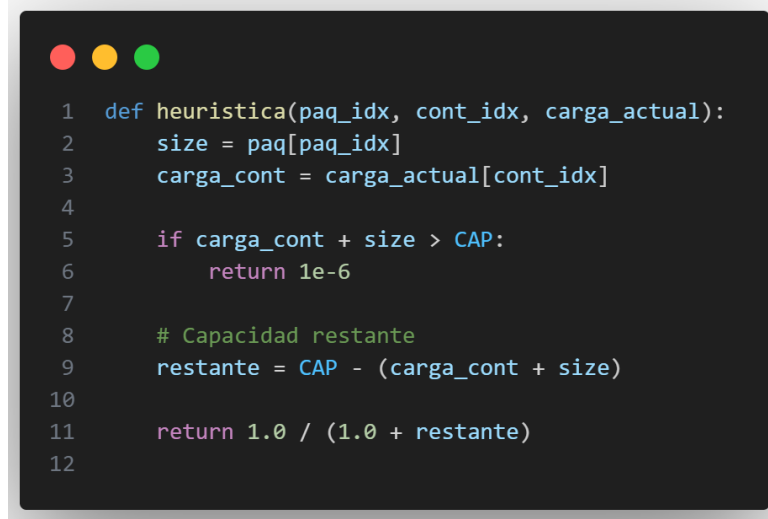
donde cada entrada τ_{ij} representa la cantidad de feromona asociada a asignar el objeto i al contenedor j . Inicialmente se suele utilizar un valor constante:

$$\tau_{ij} = \tau_0, \quad \forall i, j,$$

para no sesgar la búsqueda hacia ningún contenedor específico.

La heurística η_{ij} se define en función del espacio restante en el contenedor j si se colocara el objeto i . Como se menciona en la ecuación 2

Esto favorece asignaciones que llenan mejor el contenedor sin exceder la capacidad. Esta heurística se recalcula dinámicamente mientras la hormiga construye su solución, ya que el espacio disponible cambia conforme se asignan objetos.



```

1  def heuristica(paq_idx, cont_idx, carga_actual):
2      size = paq[paq_idx]
3      carga_cont = carga_actual[cont_idx]
4
5      if carga_cont + size > CAP:
6          return 1e-6
7
8      # Capacidad restante
9      restante = CAP - (carga_cont + size)
10
11     return 1.0 / (1.0 + restante)
12

```

Figura 3: Funcion heuristica utilizada en python

2.4. Construcción de soluciones por las hormigas

En cada iteración, cada hormiga construye una solución asignando los objetos uno a uno. Para cada objeto i , la hormiga evalúa los contenedores factibles (aquellos donde el objeto cabe sin exceder la capacidad) y selecciona un contenedor j de acuerdo con la probabilidad:

$$P_{ij} = \frac{\tau_{ij}^{\alpha} \eta_{ij}^{\beta}}{\sum_{l \in \text{Contenedores válidos}} \tau_{il}^{\alpha} \eta_{il}^{\beta}}. \quad (6)$$

Esta regla combina la información histórica (feromonas) con la información local (heurística).

```

1 def construir_solucion(tau, alpha, beta):
2     carga = [0] * n_contenedores
3     solucion = [None] * n_paquetes
4
5     for j in range(n_paquetes):
6         atractividades = []
7         for k in range(n_contenedores):
8             eta = heuristica(j, k, carga)
9             t = tau[j, k] ** alpha
10            a = t * (eta ** beta)
11            atractividades.append(a)
12
13        atractividades = np.array(atractividades)
14        probs = atractividades / atractividades.sum()
15
16        cont_elegido = np.random.choice(range(n_contenedores), p=probs)
17
18        solucion[j] = cont_elegido
19        carga[cont_elegido] += paq[j]
20
21    return solucion

```

Figura 4: Función Construcción de Solución utilizada en python

2.5. Evaluación de la función de aptitud

Una vez que una hormiga ha construido su solución completa, se evalúa mediante una función de aptitud diseñada para:

- Maximizar el uso de capacidad de los contenedores empleados.
- Minimizar el número de contenedores utilizados.

Se emplea una función de la forma:

$$f = \frac{2}{N} \sum_{i=1}^N \frac{F_i}{C}, \quad (7)$$

donde N es el número de contenedores usados, F_i es la suma de tamaños de los objetos en el contenedor i y C es la capacidad de cada contenedor. Esta función está normalizada de manera que valores cercanos a 1 indican soluciones donde los contenedores están muy bien llenos (con poco espacio desperdiciado).

Además, se pueden añadir penalizaciones si algún contenedor excede su capacidad o si se supera un número máximo de contenedores.

```

1 def eval_sol(solucion):
2     # Carga en cada contenedor
3     carga = [0] * n_contenedores
4
5     for i, cont in enumerate(solucion):
6         carga[cont] += paq[i]
7
8     # Verificar Capacidad
9     for c in carga:
10         if c > CAP:
11             return 1e-6, carga # Penalización si se excede la capacidad
12
13     # Contenedores usados
14     usados = sum(1 for c in carga if c > 0)
15
16     # Espacio desperdiciado
17     desperdicio = usados * CAP - sum(paq)
18     costo = usados + 0.1 * desperdicio
19     fitness = 1.0 / (1.0 + costo)
20
21     return fitness, carga

```

Figura 5: Funcion Evaluación de Solución utilizada en python

2.6. Actualización de feromonas

Al finalizar la construcción y evaluación de todas las soluciones en una iteración, se actualiza la matriz de feromonas en dos etapas:

1. Evaporación global:

$$\tau_{ij} \leftarrow (1 - \rho) \tau_{ij}, \quad (8)$$

con $\rho = 0,5$ como tasa de evaporación. Esto evita que las rutas antiguas dominen la búsqueda.

2. Depósito de feromonas: se refuerzan las asignaciones objeto–contenedor pertenecientes a la mejor solución (ya sea la mejor de la iteración o la mejor global):

$$\tau_{ij} \leftarrow \tau_{ij} + \Delta\tau_{ij}, \quad (9)$$

donde

$$\Delta\tau_{ij} = \begin{cases} \frac{Q}{f_{\text{best}}}, & \text{si la asignación } (i, j) \text{ está en la mejor solución,} \\ 0, & \text{en otro caso,} \end{cases} \quad (10)$$

y $Q = 1,0$ es la cantidad base de feromona depositada, mientras que f_{best} es la aptitud de la mejor solución.

La lógica de actualización se implementa en una función auxiliar que recibe la solución ganadora y la matriz de feromonas, aplicando evaporación y depósito de forma vectorizada cuando es posible.

2.7. Ejecución del algoritmo y recolección de métricas

Por ultimo corremos nuestro algoritmo de colonia de hormigas asi como una funcion auxiliar para recolectar las metricas en cada iteración.

```
1 def ant_colony(n_hormigas, n_iteraciones):
2     # Inicializamos métricas y feromonas
3     (tau,
4     mejor_solucion_global,
5     mejor_fitness_global,
6     mejor_carga_global,
7     hist_mejor,
8     hist_prom) = metricas()
9
10    for it in range(n_iteraciones):
11        soluciones = []
12        fitnesses = []
13
14        for _ in range(n_hormigas):
15            sol = construir_solucion(tau, alpha, beta)
16            fit, carga = eval_sol(sol)
17            soluciones.append((sol, carga))
18            fitnesses.append(fit)
19
20        fitnesses = np.array(fitnesses)
21
22        # Mejor hormiga
23        idx_best = int(np.argmax(fitnesses))
24        sol_best, carga_best = soluciones[idx_best]
25        fit_best = float(fitnesses[idx_best])
26
27        # Promedio
28        fit_prom = float(fitnesses.mean())
29
30        # Guardar métricas
31        hist_mejor.append(fit_best)
32        hist_prom.append(fit_prom)
33
34        # Actualizar mejor global
35        if fit_best > mejor_fitness_global:
36            mejor_fitness_global = fit_best
37            mejor_solucion_global = sol_best
38            mejor_carga_global = carga_best
39
40        # Evaporación
41        tau = (1 - rho) * tau
42
43        # Depósito de feromona
44        for j, cont in enumerate(sol_best):
45            tau[j, cont] += Q * fit_best
46
47        # Monitoreo
48        if (it + 1) % 10 == 0 or it == 0:
49            print(
50                f"Iteración (it+1) | "
51                f"Mejor fitness iteración: {fit_best:.4f} | "
52                f"Promedio: {fit_prom:.4f} | "
53                f"Mejor global: {mejor_fitness_global:.4f}"
54            )
55
56    return mejor_solucion_global, mejor_fitness_global, mejor_carga_global, tau, hist_mejor, hist_prom
```

Figura 6: Funcion Ant Colony utilizada en python

```
1 def metricas():
2     tau = np.ones((n_paquetes, n_contenedores))
3     mejor_solucion_global = None
4     mejor_fitness_global = -1
5     mejor_carga_global = None
6
7     # Historial
8     hist_mejor = []
9     hist_prom = []
10
11    return tau, mejor_solucion_global, mejor_fitness_global, mejor_carga_global, hist_mejor, hist_prom
```

Figura 7: Funcion Metricas utilizada en python

En cada iteración se ejecutan los siguientes pasos:

1. Inicializar una lista vacía de soluciones y aptitudes.

2. Para cada hormiga:
 - a)* Construir una solución completa (Figura 4).
 - b)* Evaluar su aptitud (Figura 5).
3. Actualizar la mejor solución global si alguna hormiga la mejora.
4. Actualizar las feromonas.
5. Llamar a la función *metricas* (Figura 7) para registrar:
 - La mejor aptitud de la iteración.
 - El promedio de aptitud de todas las hormigas.
 - Información adicional sobre la carga de los contenedores.

3. Resultados

4. Discusión

5. Conclusión

6. Bibliografía

Referencias

- [1] Marco Dorigo, Vittorio Maniezzo y Alberto Coloni. «Ant system: Optimization by a colony of cooperating agents». En: *IEEE Transactions on Systems, Man, and Cybernetics—Part B* 26.1 (1996), págs. 29-41.
- [2] Marco Dorigo y Thomas Stützle. *Ant Colony Optimization*. MIT Press, 2006.